

docker_lab

Guia de instalação e uso — execução de experimentos com CPU, RAM e GPU controlados em servidor compartilhado

Projeto: github.com/lucas-mendonca-andrade/docker_lab

1. Para que serve

Vários alunos usam a mesma máquina de laboratório. Sem controle, um único experimento pesado podia esgotar CPU, memória ou GPU, derrubar o acesso remoto de todos e até reiniciar a máquina. O `docker_lab` resolve isso com três mecanismos:

Execução em container

Todo script Python roda dentro de um container Docker, com a quantidade de CPU, RAM e VRAM que o próprio aluno pediu, e nada além disso.

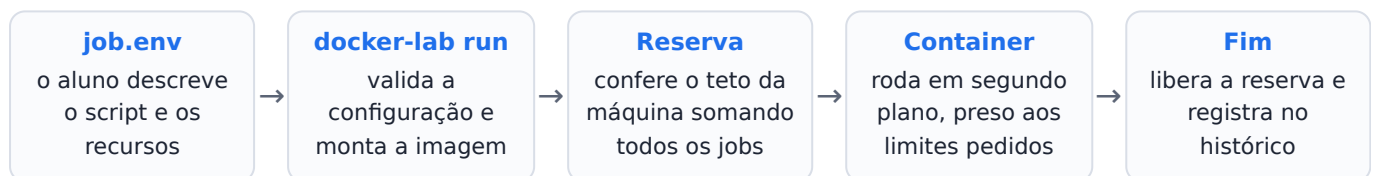
Teto da máquina

A soma do que todos os jobs reservaram nunca passa de 80% dos cores e da VRAM, nem de 90% da RAM. Um pedido que ultrapassaria o teto é recusado na hora.

Proteções na máquina

A potência da GPU fica limitada a 80%, e rodar `python3` direto no terminal mostra um aviso em vez de executar.

Como uma execução acontece



Recurso	Limite por job (definido no <code>job.env</code>)	Teto conjunto (todos os jobs)
CPU	<code>CORES</code> : quantidade de cores	80% dos cores da máquina
RAM	<code>MEMORY_GB</code> : passou disso, o job é encerrado	90% da RAM
VRAM (memória da GPU)	<code>GPU_MEMORY_GB</code> : passou disso, o job recebe <code>CUDA out of memory</code>	80% da VRAM de cada GPU
Potência da GPU	não é definida por job	80% do limite atual da GPU (TITAN V: 250 W → 200 W)

O processamento da GPU não é reservado por job: quando dois jobs usam a mesma GPU, o driver da NVIDIA alterna entre eles, e cada um fica proporcionalmente mais lento.

2. Instalação (administrador, uma única vez)

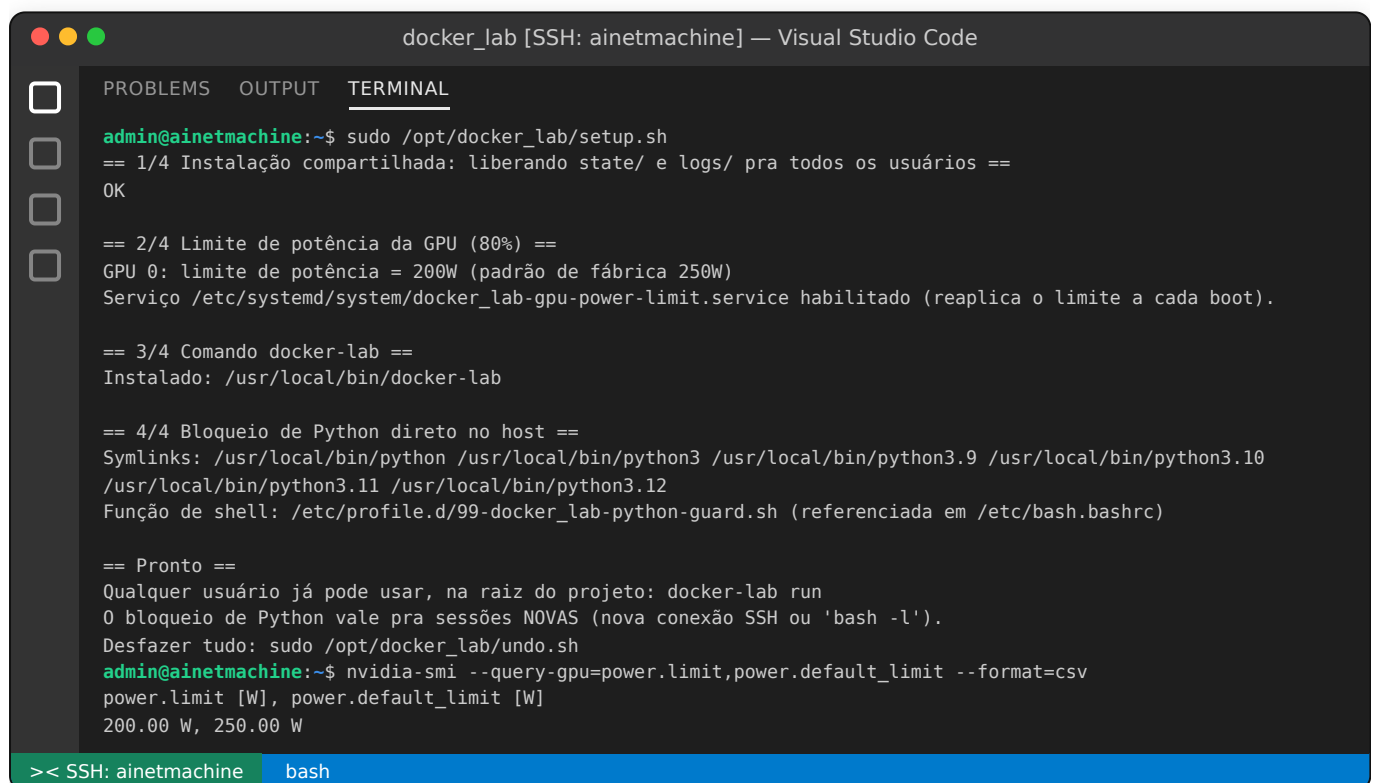
A instalação é feita na máquina do laboratório por quem tem `sudo`. São dois comandos:

```
sudo git clone https://github.com/lucas-mendonca-andrade/docker_lab.git /opt/docker_lab
sudo /opt/docker_lab/setup.sh
```

Instale em `/opt/docker_lab`, como root, e não mova a pasta depois. Se a pasta ficar na home de um aluno, os outros usuários não conseguem acessá-la, e esse aluno poderia alterar scripts que rodam como root. Para mudar de lugar: `undo.sh`, mover a pasta e rodar `setup.sh` de novo.

O `setup.sh` faz quatro coisas, e pode ser rodado de novo sem problema (por exemplo, depois de um `git pull`):

Passo	O que faz
1. Instalação compartilhada	Libera <code>state/</code> e <code>logs/</code> para todos os usuários, que passam a gravar no mesmo controle de reservas. Usuários criados no futuro já funcionam, sem nenhum comando extra.
2. Potência da GPU	Limita cada GPU a 80% do limite de potência atual e instala um serviço do <code>systemd</code> que reaplica o limite a cada boot.
3. Comando <code>docker-lab</code>	Instala o comando <code>docker-lab</code> para todos os usuários.
4. Bloqueio de Python direto	Faz <code>python3 script.py</code> no terminal mostrar um aviso em vez de executar, inclusive com <code>venv</code> ativado. Os binários do sistema não são alterados.



```
docker_lab [SSH: ainetmachine] — Visual Studio Code
PROBLEMS OUTPUT TERMINAL
admin@ainetmachine:~$ sudo /opt/docker_lab/setup.sh
== 1/4 Instalação compartilhada: liberando state/ e logs/ pra todos os usuários ==
OK

== 2/4 Limite de potência da GPU (80%) ==
GPU 0: limite de potência = 200W (padrão de fábrica 250W)
Serviço /etc/systemd/system/docker_lab-gpu-power-limit.service habilitado (reaplica o limite a cada boot).

== 3/4 Comando docker-lab ==
Instalado: /usr/local/bin/docker-lab

== 4/4 Bloqueio de Python direto no host ==
Symlinks: /usr/local/bin/python /usr/local/bin/python3 /usr/local/bin/python3.9 /usr/local/bin/python3.10
/usr/local/bin/python3.11 /usr/local/bin/python3.12
Função de shell: /etc/profile.d/99-docker_lab-python-guard.sh (referenciada em /etc/bash.bashrc)

== Pronto ==
Qualquer usuário já pode usar, na raiz do projeto: docker-lab run
O bloqueio de Python vale pra sessões NOVAS (nova conexão SSH ou 'bash -l').
Desfazer tudo: sudo /opt/docker_lab/undo.sh
admin@ainetmachine:~$ nvidia-smi --query-gpu=power.limit,power.default_limit --format=csv
power.limit [W], power.default_limit [W]
200.00 W, 250.00 W

>< SSH: ainetmachine bash
```

Figura 1 — Instalação pelo terminal do VS Code conectado à máquina do laboratório (Remote-SSH), com a conferência do limite de potência no final.

Quem estava conectado por SSH durante a instalação só passa a ter o bloqueio completo de Python depois de reconectar. **Para desfazer tudo:** `sudo /opt/docker_lab/undo.sh`. Ele remove o bloqueio, volta a GPU à potência de fábrica, remove o comando `docker-lab` e devolve as permissões originais. Histórico e logs são preservados.

3. Executando um experimento (cada aluno)

Passo 1 — Criar o `job.env` na raiz do projeto

```
cp /opt/docker_lab/job.env.example ~/meu_projeto/job.env
```

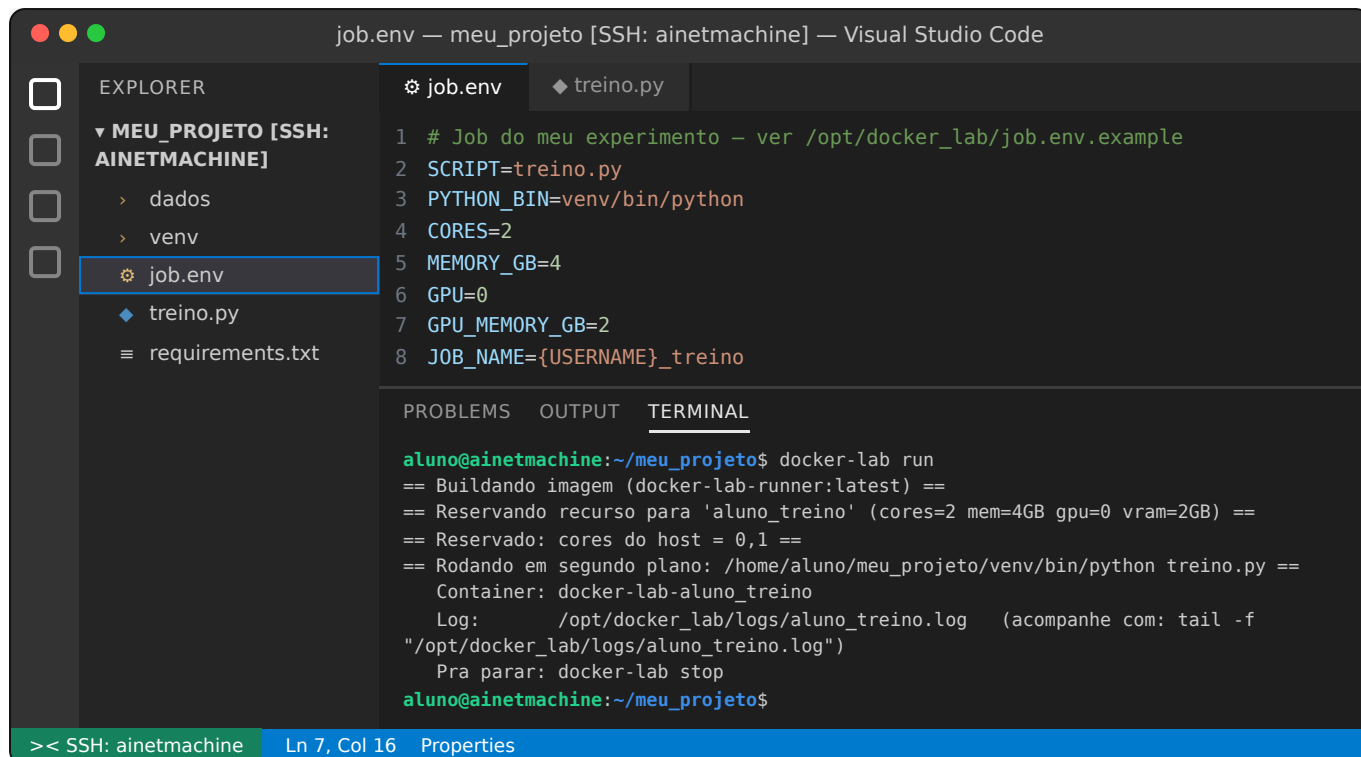
O `job.env` fica **sempre na raiz do projeto**. A pasta onde ele está é montada dentro do container, e os caminhos `SCRIPT` e `PYTHON_BIN` são relativos a ela.

Variável	Exemplo	Significado
<code>SCRIPT</code>	<code>treino.py</code>	Script a executar, relativo à raiz do projeto
<code>PYTHON_BIN</code>	<code>venv/bin/python</code>	Python usado. Normalmente é o venv do próprio projeto; são aceitos venvs de Python 3.9 a 3.12.
<code>CORES</code>	<code>2</code>	Quantidade de cores. O <code>docker_lab</code> escolhe quais cores livres usar.
<code>MEMORY_GB</code>	<code>4</code>	RAM máxima do job
<code>GPU</code>	<code>none</code> ou <code>0</code>	Não usar GPU (<code>none</code>) ou usar a GPU de índice 0, a primeira coluna do <code>nvidia-smi</code>
<code>GPU_MEMORY_GB</code>	<code>2</code>	VRAM máxima (obrigatório com GPU)
<code>JOB_NAME</code>	<code>{USERNAME}_treino</code>	Nome do job. Mantenha o <code>{USERNAME}_</code> , que é trocado pelo seu usuário automaticamente.

O `nvidia-smi` mostra cerca de 0,3 GB a mais que o `GPU_MEMORY_GB` por job: é a memória fixa que o driver reserva para cada processo (contexto CUDA), e o teto de 80% já deixa folga para ela. Não sabe quanta VRAM o seu script usa? Rode uma vez e veja o pico: `torch.cuda.max_memory_allocated()` no PyTorch, ou `nvidia-smi` durante a execução.

Passo 2 — Executar com `docker-lab run`

Abra o terminal do VS Code na pasta do projeto e rode `docker-lab run`. A validação e a reserva aparecem na tela, o job segue em segundo plano e o prompt volta na hora. Você pode fechar o VS Code ou desconectar, e o job continua rodando.



The screenshot shows the Visual Studio Code interface with a file explorer on the left and a terminal at the bottom. The file explorer shows a project named 'MEU_PROJETO [SSH: AINETMACHINE]' with files 'dados', 'venv', 'job.env', 'treino.py', and 'requirements.txt'. The 'job.env' file is open in the editor, showing configuration for a Docker container. The terminal window shows the command 'docker-lab run' being executed, which builds an image, reserves resources, and runs the 'treino.py' script in a container named 'docker-lab-aluno_treino'.

```
job.env — meu_projeto [SSH: ainetmachine] — Visual Studio Code

EXPLORER
  MEU_PROJETO [SSH: AINETMACHINE]
    dados
    venv
    job.env
    treino.py
    requirements.txt

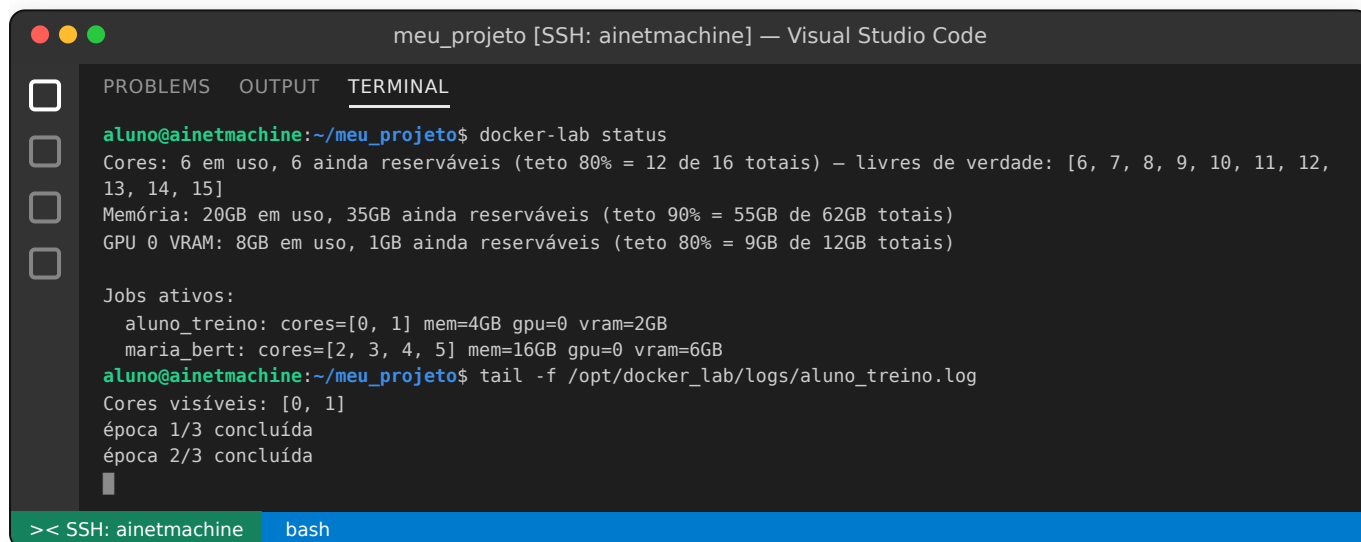
job.env
1 # Job do meu experimento – ver /opt/docker_lab/job.env.example
2 SCRIPT=treino.py
3 PYTHON_BIN=venv/bin/python
4 CORES=2
5 MEMORY_GB=4
6 GPU=0
7 GPU_MEMORY_GB=2
8 JOB_NAME={USERNAME}_treino

TERMINAL
aluno@ainetmachine:~/meu_projeto$ docker-lab run
== Buildando imagem (docker-lab-runner:latest) ==
== Reservando recurso para 'aluno_treino' (cores=2 mem=4GB gpu=0 vram=2GB) ==
== Reservado: cores do host = 0,1 ==
== Rodando em segundo plano: /home/aluno/meu_projeto/venv/bin/python treino.py ==
Container: docker-lab-aluno_treino
Log:      /opt/docker_lab/logs/aluno_treino.log   (acompanhe com: tail -f
"/opt/docker_lab/logs/aluno_treino.log")
Pra parar: docker-lab stop
aluno@ainetmachine:~/meu_projeto$
```

Figura 2 — `job.env` aberto no editor e o job disparado pelo terminal integrado. Com vários arquivos (por exemplo `job_1.env`, `job_2.env`), use `docker-lab run job_1.env`.

Passo 3 — Acompanhar: `docker-lab status` e o log

`docker-lab status` mostra o que cada recurso tem em uso, o que ainda pode ser reservado dentro do teto da máquina e quais jobs estão ativos. O andamento do script fica no log.



The screenshot shows the Visual Studio Code interface with the terminal window displaying the output of 'docker-lab status' and 'tail -f'. The status output shows resource usage (Cores, Memória, GPU VRAM) and active jobs. The tail output shows the progress of the training script, indicating that the first epoch is complete and the second epoch is in progress.

```
meu_projeto [SSH: ainetmachine] — Visual Studio Code

TERMINAL
aluno@ainetmachine:~/meu_projeto$ docker-lab status
Cores: 6 em uso, 6 ainda reserváveis (teto 80% = 12 de 16 totais) – livres de verdade: [6, 7, 8, 9, 10, 11, 12, 13, 14, 15]
Memória: 20GB em uso, 35GB ainda reserváveis (teto 90% = 55GB de 62GB totais)
GPU 0 VRAM: 8GB em uso, 1GB ainda reserváveis (teto 80% = 9GB de 12GB totais)

Jobs ativos:
  aluno_treino: cores=[0, 1] mem=4GB gpu=0 vram=2GB
  maria_bert: cores=[2, 3, 4, 5] mem=16GB gpu=0 vram=6GB
aluno@ainetmachine:~/meu_projeto$ tail -f /opt/docker_lab/logs/aluno_treino.log
Cores visíveis: [0, 1]
época 1/3 concluída
época 2/3 concluída
```

Figura 3 — Dois alunos dividindo a mesma GPU (TITAN V, 12 GB): cada um tem a sua fatia de VRAM, e a soma respeita o teto de 9 GB. O `Ctrl+C` sai do `tail` sem parar o job.

O job roda com o **seu usuário**: tudo o que o script gravar no projeto (resultados, modelos, logs) é seu, e você pode apagar ou editar sem `sudo`. Só instalar e desinstalar o `docker_lab` exige `sudo`.

Passo 4 — Parar um job e ver o histórico

`docker-lab stop` lista os jobs ativos num menu numerado. Parar um job **encerra** a execução, que não pode ser retomada depois. A reserva é liberada sozinha. `docker-lab history` mostra todas as execuções com início, fim e status: `executando`, `executou`, `erro` ou `interrompido`.

```
meu_projeto [SSH: ainetmachine] — Visual Studio Code

PROBLEMS OUTPUT TERMINAL

aluno@ainetmachine:~/meu_projeto$ docker-lab stop
Jobs ativos:
1) aluno_treino (cores=[0, 1] mem=4GB gpu=0 vram=2GB)
2) maria_bert (cores=[2, 3, 4, 5] mem=16GB gpu=0 vram=6GB)
3) Cancelar
Qual você quer parar (número, ou 'Cancelar')? 1
Parando 'docker-lab-aluno_treino'...
docker-lab-aluno_treino
Parado. A reserva é liberada sozinha em alguns instantes (o run.sh cuida disso).
aluno@ainetmachine:~/meu_projeto$ docker-lab history
JOB_NAME          USUARIO    INICIO          FIM             STATUS
-----
aluno_teste_cpu   aluno      2026-09-28T09:12:40 2026-09-28T09:13:41 executou
maria_bert        maria      2026-09-28T10:02:11 -                executando
aluno_treino      aluno      2026-09-28T10:30:05 2026-09-28T10:41:52 interrompido

>< SSH: ainetmachine bash
```

Figura 4 — Parando um job pelo menu e conferindo o histórico. Qualquer usuário pode parar qualquer job, se precisar.

4. O que o docker_lab impede

Rodar Python direto, fora do container

Com o bloqueio ativo, `python3 script.py` no terminal mostra instruções em vez de executar, inclusive com o venv ativado. A sessão do terminal continua normal.

```
treino.py — meu_projeto [SSH: ainetmachine] — Visual Studio Code

EXPLORER
▼ MEU_PROJETO [SSH: AINETMACHINE]
  > dados
  > venv
  ⚙ job.env
  ◆ treino.py
  ≡ requirements.txt

job.env
◆ treino.py
1 import torch
2 from modelo import treinar
3
4 treinar(epocas=3, device="cuda")

PROBLEMS OUTPUT TERMINAL

(venv) aluno@ainetmachine:~/meu_projeto$ python3 treino.py
[docker_lab] Execução direta de Python bloqueada nesta máquina.

Scripts Python devem rodar DENTRO de um container gerenciado pelo docker_lab, que
garante limite de CPU/RAM/GPU e evita travar a máquina pros outros usuários.

Como rodar:
1. Copie o exemplo pra raiz do SEU projeto (o job.env sempre fica lá):
cp /opt/docker_lab/job.env.example <raiz-do-seu-projeto>/job.env
2. Edite job.env (SCRIPT, PYTHON_BIN, CORES, MEMORY_GB, GPU, GPU_MEMORY_GB,
JOB_NAME).
3. Na raiz do seu projeto, rode:
docker-lab run

Mais detalhes: /opt/docker_lab/README.md
Projeto no GitHub: https://github.com/lucas-mendonca-andrade/docker_lab
(venv) aluno@ainetmachine:~/meu_projeto$

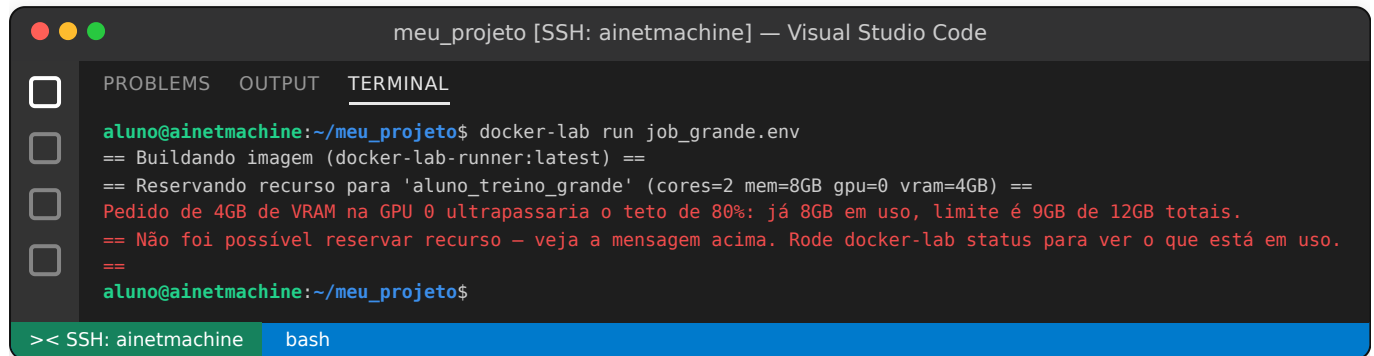
>< SSH: ainetmachine Python 3.11.9 ('venv')
```

Figura 5 — Tentativa de rodar o script direto, mesmo com o venv ativado: aparece o aviso com o caminho correto.

O bloqueio é uma **barreira contra esquecimento**, não uma trava de segurança. Caminhos absolutos (`/usr/bin/python3.11 x.py`), `./script.py` e scripts `.sh` não interativos passam sem aviso. Use sempre o `docker-lab run`.

Passar do teto da máquina

Se o pedido de um job, somado ao que já está reservado, passar do teto de qualquer recurso, a execução é recusada na hora, com a explicação do motivo. Nada entra em fila: espere algum job terminar, ou peça menos recurso.



```
meu_projeto [SSH: ainetmachine] — Visual Studio Code
PROBLEMS OUTPUT TERMINAL
aluno@ainetmachine:~/meu_projeto$ docker-lab run job_grande.env
== Buildando imagem (docker-lab-runner:latest) ==
== Reservando recurso para 'aluno_treino_grande' (cores=2 mem=8GB gpu=0 vram=4GB) ==
Pedido de 4GB de VRAM na GPU 0 ultrapassaria o teto de 80%: já 8GB em uso, limite é 9GB de 12GB totais.
== Não foi possível reservar recurso – veja a mensagem acima. Rode docker-lab status para ver o que está em uso. ==
aluno@ainetmachine:~/meu_projeto$
>< SSH: ainetmachine bash
```

Figura 6 — Pedido de VRAM recusado: a GPU já tem 8 GB reservados, e o teto é 9 GB.

Passar do limite do próprio job

Situação	O que acontece	O que fazer
O script usa mais RAM que <code>MEMORY_GB</code>	O container é encerrado. O histórico mostra <code>erro</code> .	Aumentar <code>MEMORY_GB</code>
O script usa mais VRAM que <code>GPU_MEMORY_GB</code>	O script recebe <code>CUDA out of memory</code> , e só esse job é afetado.	Aumentar <code>GPU_MEMORY_GB</code> ou reduzir o batch size
O script pede mais cores (ex. <code>n_jobs=-1</code>)	Ele só enxerga os cores reservados e roda com eles.	Nada: é o esperado
<code>JOB_NAME</code> já em uso por um job ativo	A execução é recusada.	Usar outro nome no <code>job.env</code>

5. Resumo dos comandos

Administrador (uma vez)

```
sudo git clone https://github.com/lucas-mendonca-andrade/docker_lab.git
/opt/docker_lab
sudo /opt/docker_lab/setup.sh
# desfazer tudo:
sudo /opt/docker_lab/undo.sh
```

Aluno (na raiz do projeto)

```
cp /opt/docker_lab/job.env.example job.env
docker-lab run # ou: docker-lab run outro.env
docker-lab status
docker-lab stop
docker-lab history
docker-lab corrigir-permissoes # arquivos antigos com dono root
```

Código-fonte, README e atualizações: https://github.com/lucas-mendonca-andrade/docker_lab